

# Building Global IT Professionals **since 2008**

---

**19K+**

Certified Students

**160+**

Courses

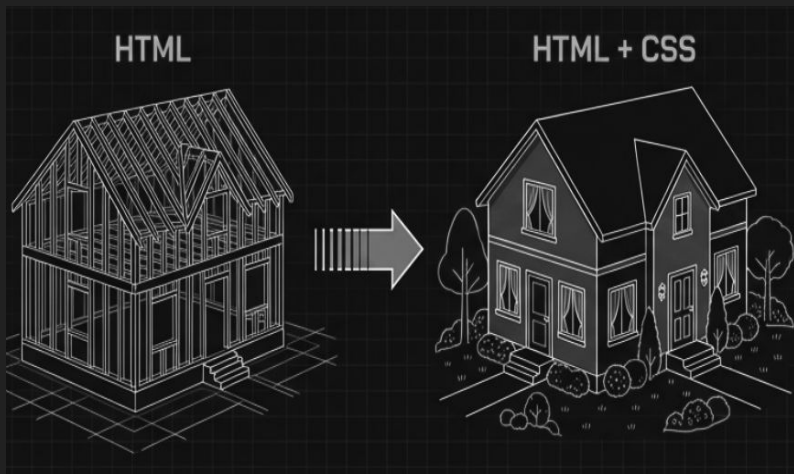
**85+**

Qualified Teachers

[www.broadwayinfosys.com](http://www.broadwayinfosys.com)

# Introduction to CSS

If HTML is the raw framing and skeleton of a house the walls, the roof, the foundational bricks—— CSS (Cascading Style Sheets) is the interior design, the paint, and the landscaping. HTML tells the browser what the content is; CSS tells the browser exactly how it should look.



```
1 <!-- HTML (The Raw Structure) -->
2 <h1>Welcome to My Website</h1>
3
4 <!-- CSS (The Paint & Design) -->
5 <style>
6   h1 {
7     color: royalblue;
8     text-align: center;
9   }
10 </style>
```

# The Anatomy of a Declarative CSS Rule

Writing CSS is like giving a specific work order to a contractor.

Every CSS rule has three vital parts:

- The **Selector** points to who you want to change.
- The **Property** specifies what feature you want to change.
- The **Value** states exactly what the new state should be.



```
/* Selector { Property: Value; } */  
  
p {  
  color: darkslategray;  
  background-color: lightgray;  
}
```

# Inline CSS - Messy

- Inline CSS is written directly inside an HTML tag using the style attribute.
- It only affects that one specific element.

**HARD TO MAINTAIN --- ???**



```
<!-- The CSS is applied directly  
inside the HTML element -->  
<h2 style="color: crimson; font-  
size: 24px;">  
    This is an urgent warning!  
</h2>
```



# Internal CSS - Single Room Decoration

- Internal CSS is placed inside a `<style>` tag, which lives inside the `<head>` of your HTML document .
- It can style anything inside that specific HTML page, but it cannot share those styles with any other pages on your website.



```
<head>
  <title>About Us</title>
  <style>
    body {
      background-color: #f4f4f4;
    }
    p {
      color: #333333;
    }
  </style>
</head>
```



# External CSS - Master Blueprint

- This is the modern industry standard for web engineering. You write all your CSS in a completely separate file (ending in .css) and link it to your HTML using the <link> tag.
- One single CSS file can control the design of thousands of HTML pages simultaneously.

HTML

```
<link rel="stylesheet" href="styles.css">
```



```
<!-- Inside your HTML <head> -->
<link rel="stylesheet"
href="styles.css">
```

```
/* Inside styles.css (a totally
separate file) */
nav {
    background-color: black;
    color: white;
}
```



# Cascade Resolves Design Conflicts

- Why is it called Cascading Style Sheets? Because styles flow downward like a waterfall.
- If multiple rules target the exact same element, the browser needs a way to decide which one wins.
- Generally, the rule with the highest specificity wins. If specificity is an exact tie, the rule written last (closest to the bottom of the cascade) overrides the earlier ones.



```
/* In this scenario, the button  
will be GREEN. */  
/* The browser reads top-to-bottom,  
so the last rule wins a tie. */
```

```
button {  
  background-color: blue;  
}
```

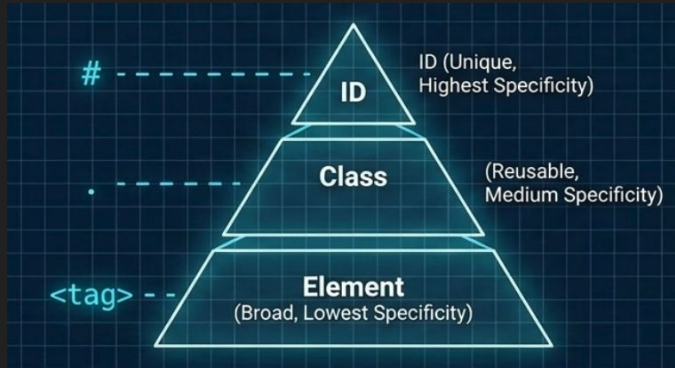
```
button {  
  background-color: green;  
}
```



# Elements: Classes vs. IDs

To target elements effectively, we use different selector types:

- Element selectors target raw HTML tags broadly.
- Class selectors (starting with a `.`) are reusable and carry moderate specificity—they are the absolute workhorse of modern CSS.
- ID selectors (starting with a `#`) carry high specificity but must be completely unique per page.



```
/* Element Selector (Broadest) */  
p { line-height: 1.5; }  
  
/* Class Selector (Reusable  
Workhorse) */  
.highlight-box { border: 2px solid  
yellow; }  
  
/* ID Selector (Unique, High  
Specificity) */  
#main-navigation {  
  background: black;  
}
```





# Fixed Sizes: Using Pixels (px)

- When sizing elements or text, we must choose a unit of measurement.
- A Pixel (px) is an Absolute Unit. It is like measuring something with a rigid wooden ruler. If you explicitly tell a box to be 300px wide, it will stay exactly 300px wide regardless of the user's screen size or default browser settings.
- Pixels are highly predictable for borders, but poor choices for accessible text sizing.

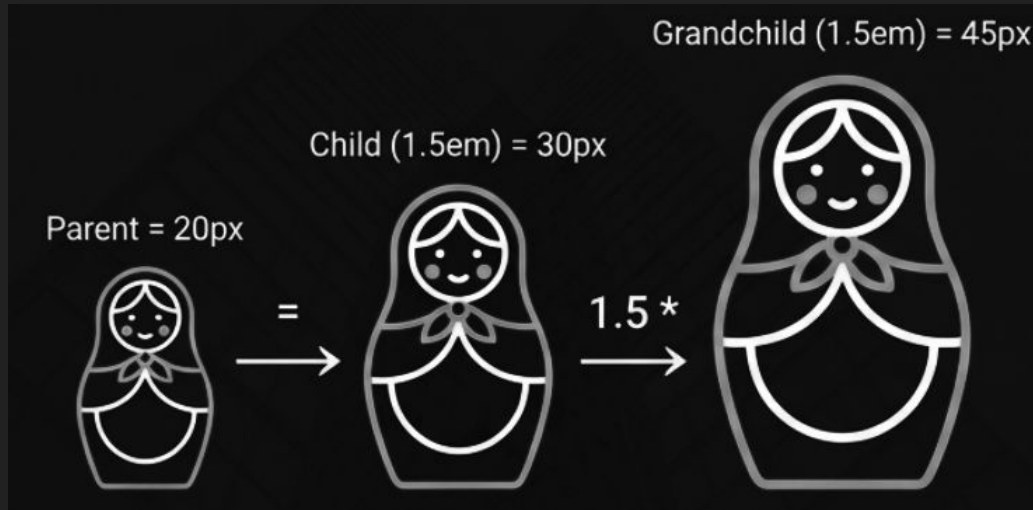


```
.profile-card {  
  width: 400px;  
  height: 250px;  
  border-width: 2px;  
}
```



# Flexible Sizes: Sizing with "em"

Relative units act like a flexible tape measure - they adapt based on their surroundings. The em unit is relative to the font size of its direct parent element.



```
<div class="parent">  
  <p class="child">I am sized  
    relative to my parent.</p>  
</div>
```

```
.parent {  
  font-size: 20px;  
}  
  
.child {  
  /* 1.5 * 20px = 30px */  
  font-size: 1.5em;  
}
```

# Consistent Flexible Sizes: Sizing with "rem"

- To fix the compounding problem of em, professional engineers use rem (Root EM).
- A rem is always relative to the root font size of the entire HTML document (usually 16px), no matter how deeply it is nested.
- It is the gold standard for accessible typography. If a user increases their browser's default font size, your entire site scales perfectly in unison.

```
html {  
  /* The global root size */  
  font-size: 16px;  
}  
  
h1 {  
  /* 3 * 16px = 48px, regardless of HTML nesting */  
  font-size: 3rem;  
}
```

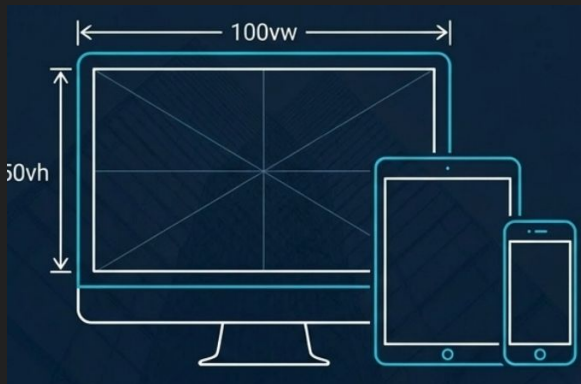


# Screen-Based Sizes: Making Layouts Fit Any Device

When building layouts that must seamlessly adapt to phones, tablets, and desktops, we use units relative to the browser window itself (the Viewport).

- 1vw is exactly 1% of the screen's total width.
- 1vh is 1% of the screen's total height.

Percentages (%) are similar, but they are relative to the parent container's size rather than the viewport.



```
.hero-banner {  
  /* Spans 100% of the browser's width */  
  width: 100vw;  
  /* Takes up exactly half of the browser's height */  
  height: 50vh;  
}  
  
.child-box {  
  /* Takes up half the width of the .hero-banner */  
  width: 50%;  
}
```



# Quick Guide: Choosing the Right Size Unit

| Unit        | Relative to...        | Best Use Case         | Accessibility  |
|-------------|-----------------------|-----------------------|----------------|
| px          | Nothing (Rigid)       | Borders, tiny details | Poor           |
| rem         | Root Document         | All Typography        | Excellent      |
| em          | Direct Parent         | Proportional Padding  | Good (Careful) |
| vh / vw / % | Viewport / Parent Box | Macro Layouts         | Neutral        |

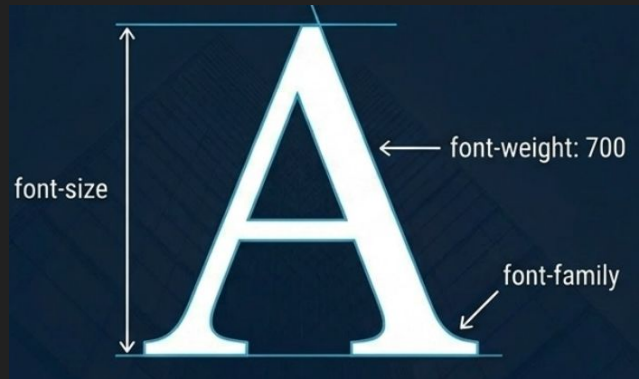


```
.perfect-button {  
  border: 2px solid black; /* Rigid px */  
  font-size: 1.2rem; /* Accessible rem */  
  padding: 0.5em 1em; /* Proportional em */  
  width: 100%; /* Responsive % */  
}
```



# Text Basics: Font Style and Thickness

- **Typography accounts for 90% of web design.**
- **The three most critical properties are font-family (the typeface you select), font-size (how large it is), and font-weight (how thick or bold the letters are).**
- **Always provide a generic fallback font (like sans-serif) to prevent layout crashes if your primary font fails to load.**

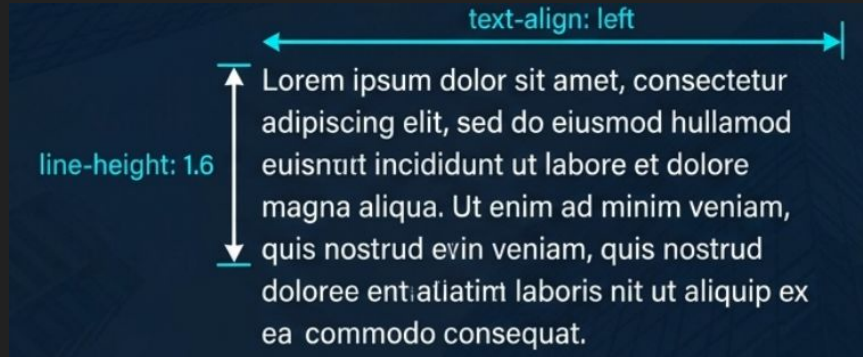


```
p {  
  /* Arial is primary, sans-serif is the  
  safety backup */  
  font-family: "Arial", sans-serif;  
  font-size: 1.1rem;  
  
  /* 400 is standard normal text, 700 is  
  bold */  
  font-weight: 700;  
}
```



# Web Text: Using Google Fonts and Alignment

- You are not limited to local fonts. The Google Fonts API is a massive library for web typography.
- You fetch these font files via an `@import` rule placed at the very top of your CSS file.
- Once imported, you refine the reading experience using `line-height` to create vertical breathing room, and `text-align` to structure the paragraph layout.



```
/* 1. Import at the very top of your CSS file */
@import url('https://fonts.googleapis.com/css2?family=
Roboto:wght@400;700&display=swap');

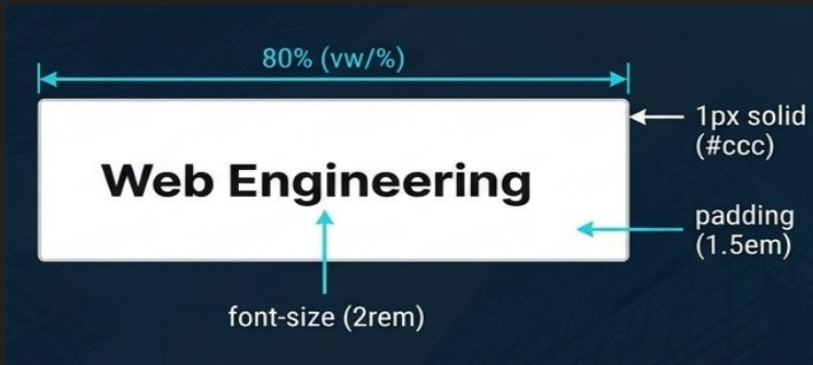
/* 2. Apply and refine */
article {
  font-family: 'Roboto', sans-serif;
  line-height: 1.6; /* Crucial vertical breathing room */
  text-align: left;
}
```





# Putting It All Together: Building a Real Component

- Let's bring all our tools together into a final blueprint.
- We will link an external stylesheet, import a custom web font, use rem for accessible text sizing, em for proportional spacing, and absolute px for borders.
- This code block represents exactly what professional, manual CSS engineering looks like in practice.



```
<link rel="stylesheet" href="main.css">

<div class="card">
  <h2>Web Engineering</h2>
</div>

@import url('https://fonts.googleapis.com/css2?
family=Inter&display=swap');

.card {
  font-family: 'Inter', sans-serif;
  width: 80%; /* Responsive layout */
  border: 1px solid #ccc; /* Rigid detail */
  padding: 1.5em; /* Proportional spacing */
}

.card h2 {
  font-size: 2rem; /* Accessible typography */
  line-height: 1.2;
}
```



# ***THANK YOU!***

---



Shree Ganesh Marg, Subidhanagar, Tinkune, Kathmandu

01-4117578 / 411849 / 9841002000

[www.broadwayinfosys.com](http://www.broadwayinfosys.com)